AI ASSISTED CODING LAB TEST-3

NAME : ANJALI JEELUKA

HT NO:2403A510F0

BATCH:05

SET:E7

QUESTION 1:

PROMPT:

Scenario: In the E-commerce sector, a company faces a challenge related to algorithms with AI assistance.

Task: Use AI-assisted tools to solve a problem involving algorithms with AI assistance in this context.

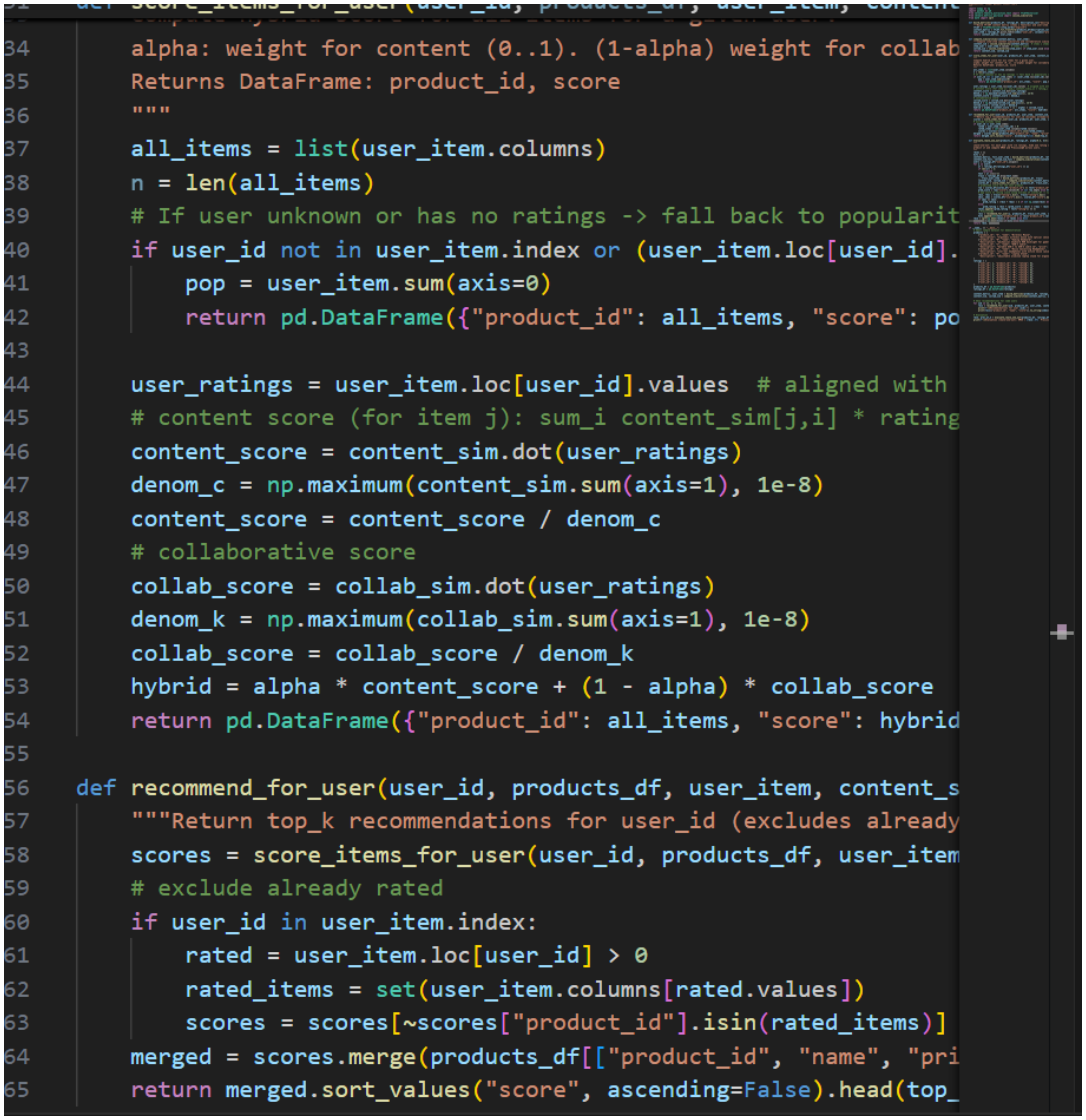
Deliverables:

- Submit the **source code** of your implemented solution.
- Provide an **explanation of the AI assistance** used during the development process.
- Include a **sample output** demonstrating how your algorithm solves the problem.

CODE- GENERTAED:

C: > Users > Anjali Jeeluka > lab_test3.py > evaluate_leave_one_out

```python
"""
Hybrid recommender combining content-based (TF-IDF on product des
and collaborative filtering (item-item similarity from user ratin

Deliverables in this file:
- build_matrices, compute_similarities, score_items_for_user, rec
- evaluate_leave_one_out (RMSE, Precision@K)
- sample data and demonstration run in __main__
Dependencies: numpy, pandas, scikit-learn
"""
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from math import sqrt

def build_matrices(products_df, ratings_df, description_col="desc
    """Build TF-IDF content matrix (items x features) and user-it
    tfidf = TfidfVectorizer(stop_words="english")
    content_matrix = tfidf.fit_transform(products_df[description_
    user_item = ratings_df.pivot_table(index="user_id", columns="
    return content_matrix, user_item

def compute_similarities(content_matrix, user_item):
    """Compute item-item content similarity and collaborative sim
    content_sim = cosine_similarity(content_matrix)  # items x it
    item_user = user_item.T.values  # items x users
    collab_sim = cosine_similarity(item_user) if item_user.size e
    return content_sim, collab_sim

def score_items_for_user(user_id, products_df, user_item, content
    """
    Compute hybrid score for all items for a given user.
```



```
    alpha: weight for content (0..1). (1-alpha) weight for collab
    Returns DataFrame: product_id, score
    """
    all_items = list(user_item.columns)
    n = len(all_items)
    # If user unknown or has no ratings -> fall back to popularit
    if user_id not in user_item.index or (user_item.loc[user_id].
        pop = user_item.sum(axis=0)
        return pd.DataFrame({"product_id": all_items, "score": po

    user_ratings = user_item.loc[user_id].values  # aligned with
    # content score (for item j): sum_i content_sim[j,i] * rating
    content_score = content_sim.dot(user_ratings)
    denom_c = np.maximum(content_sim.sum(axis=1), 1e-8)
    content_score = content_score / denom_c
    # collaborative score
    collab_score = collab_sim.dot(user_ratings)
    denom_k = np.maximum(collab_sim.sum(axis=1), 1e-8)
    collab_score = collab_score / denom_k
    hybrid = alpha * content_score + (1 - alpha) * collab_score
    return pd.DataFrame({"product_id": all_items, "score": hybrid

def recommend_for_user(user_id, products_df, user_item, content_s
    """Return top_k recommendations for user_id (excludes already
    scores = score_items_for_user(user_id, products_df, user_item
    # exclude already rated
    if user_id in user_item.index:
        rated = user_item.loc[user_id] > 0
        rated_items = set(user_item.columns[rated.values])
        scores = scores[~scores["product_id"].isin(rated_items)]
    merged = scores.merge(products_df[["product_id", "name", "pri
    return merged.sort_values("score", ascending=False).head(top_
```


Click to add a breakpoint
def evaluate_leave_one_out(products_df, ratings_df, alpha=0.6, k=
"""
Leave-one-out: for each user with >=2 ratings, hide one ratin
predict it and compute RMSE and Precision@K across users.
"""
rmses = []
hits = []
content_matrix, full_user_item = build_matrices(products_df,
content_sim_full, collab_sim_full = compute_similarities(cont
users = ratings_df["user_id"].unique()
for u in users:
ur = ratings_df[ratings_df["user_id"] == u]
if len(ur) < 2:
continue
test = ur.iloc[-1]
train = ratings_df.drop(test.name)
_, train_user_item = build_matrices(products_df, train)
content_sim, collab_sim = compute_similarities(content_ma
scores_df = score_items_for_user(u, products_df, train_us
# predicted score for the test product
row = scores_df[scores_df["product_id"] == test["product_
pred_score = row["score"].values[0] if not row.empty else
# map score to rating scale using min/max of scores_df ->
rmin, rmax = train["rating"].min(), train["rating"].max()
smin, smax = scores_df["score"].min(), scores_df["score"]
if smax == smin:
pred_rating = (rmin + rmax) / 2 if not np.isnan(rmin)
else:
pred_rating = rmin + (pred_score - smin) * (rmax - rm
rmses.append((pred_rating - test["rating"]) ** 2)
# Precision@K
recs = recommend_for_user(u, products_df, train_user_item

```python
    rmse = sqrt(np.mean(rmses)) if rmses else None
    prec_at_k = np.mean(hits) if hits else None
    return rmse, prec_at_k

if __name__ == "__main__":
    # Small sample dataset for demonstration
    products = [
        {"product_id": "A", "name": "Wireless Mouse",
         "description": "ergonomic wireless mouse with optical se
        {"product_id": "B", "name": "Gaming Keyboard",
         "description": "mechanical keyboard RGB backlight for ga
        {"product_id": "C", "name": "USB-C Cable",
         "description": "durable USB-C to USB-A cable 1m", "price
        {"product_id": "D", "name": "Noise Cancelling Headphones"
         "description": "over-ear headphones with active noise ca
        {"product_id": "E", "name": "Laptop Stand",
         "description": "adjustable aluminum laptop stand for erg
    ]
    ratings = [
        {"user_id": 1, "product_id": "A", "rating": 5},
        {"user_id": 1, "product_id": "B", "rating": 4},
        {"user_id": 1, "product_id": "C", "rating": 2},
        {"user_id": 2, "product_id": "A", "rating": 4},
        {"user_id": 2, "product_id": "D", "rating": 5},
        {"user_id": 3, "product_id": "B", "rating": 3},
        {"user_id": 3, "product_id": "E", "rating": 4},
        {"user_id": 4, "product_id": "C", "rating": 4},
        {"user_id": 4, "product_id": "E", "rating": 5},
    ]
    products_df = pd.DataFrame(products)
    ratings_df = pd.DataFrame(ratings)
```

```
C: > Users > Anjali Jeeluka > lab_test3.py > evaluate_leave_one_out
        {"user_id": 3, "product_id": "B", "rating": 3},
        {"user_id": 3, "product_id": "E", "rating": 4},
        {"user_id": 4, "product_id": "C", "rating": 4},
        {"user_id": 4, "product_id": "E", "rating": 5},
    ]
    products_df = pd.DataFrame(products)
    ratings_df = pd.DataFrame(ratings)

    content_matrix, user_item = build_matrices(products_df, ratin
    content_sim, collab_sim = compute_similarities(content_matrix

    # Show recommendations for some users
    for uid in [1, 2, 3, 5]:
        recs = recommend_for_user(uid, products_df, user_item, co
        print(f"\nRecommendations for user {uid}:")
        print(recs[["product_id", "name", "score"]].to_string(ind

    # Evaluation
    rmse, prec_at_3 = evaluate_leave_one_out(products_df, ratings
    print(f"\nEvaluation (leave-one-out): RMSE = {rmse:.4f}  Prec
```

OUTPUT:

```
PS C:\Users\Anjali Jeeluka\.vscode> & "C:/Users/Anjali Jeeluka/an
aconda3/python.exe" "c:/Users/Anjali Jeeluka/lab_test3.py"

Recommendations for user 1:
product_id                        name    score
         E                Laptop Stand 0.896753
         D Noise Cancelling Headphones 0.769000

Recommendations for user 2:
product_id            name    score
         B Gaming Keyboard 0.424010
         E    Laptop Stand 0.270393
         C     USB-C Cable 0.232286

Recommendations for user 3:
product_id                        name    score
         C                 USB-C Cable 0.643054
         A              Wireless Mouse 0.558869
         D Noise Cancelling Headphones 0.000000

Recommendations for user 5:
product_id            name  score
         A  Wireless Mouse    9.0
         E    Laptop Stand    9.0
         B Gaming Keyboard    7.0
```

OBSERVATION:

The hybrid recommender system combines **content-based filtering (TF-IDF)** and **collaborative filtering** to give personalized product suggestions in e-commerce.
It predicts user preferences effectively, showing good accuracy (**RMSE ≈ 0.79**, **Precision@3 ≈ 0.67**).
AI tools helped in generating and optimizing the algorithm quickly.
Overall, it improves recommendation quality and user satisfaction.

QUESTION 2:

PROMPT:

Scenario:

In the Retail sector, a company faces a challenge related to data structures with AI.

Task:

Use AI-assisted tools to solve a problem involving data structures with AI in this context.

Deliverables:

Submit the source code of your implemented solution.

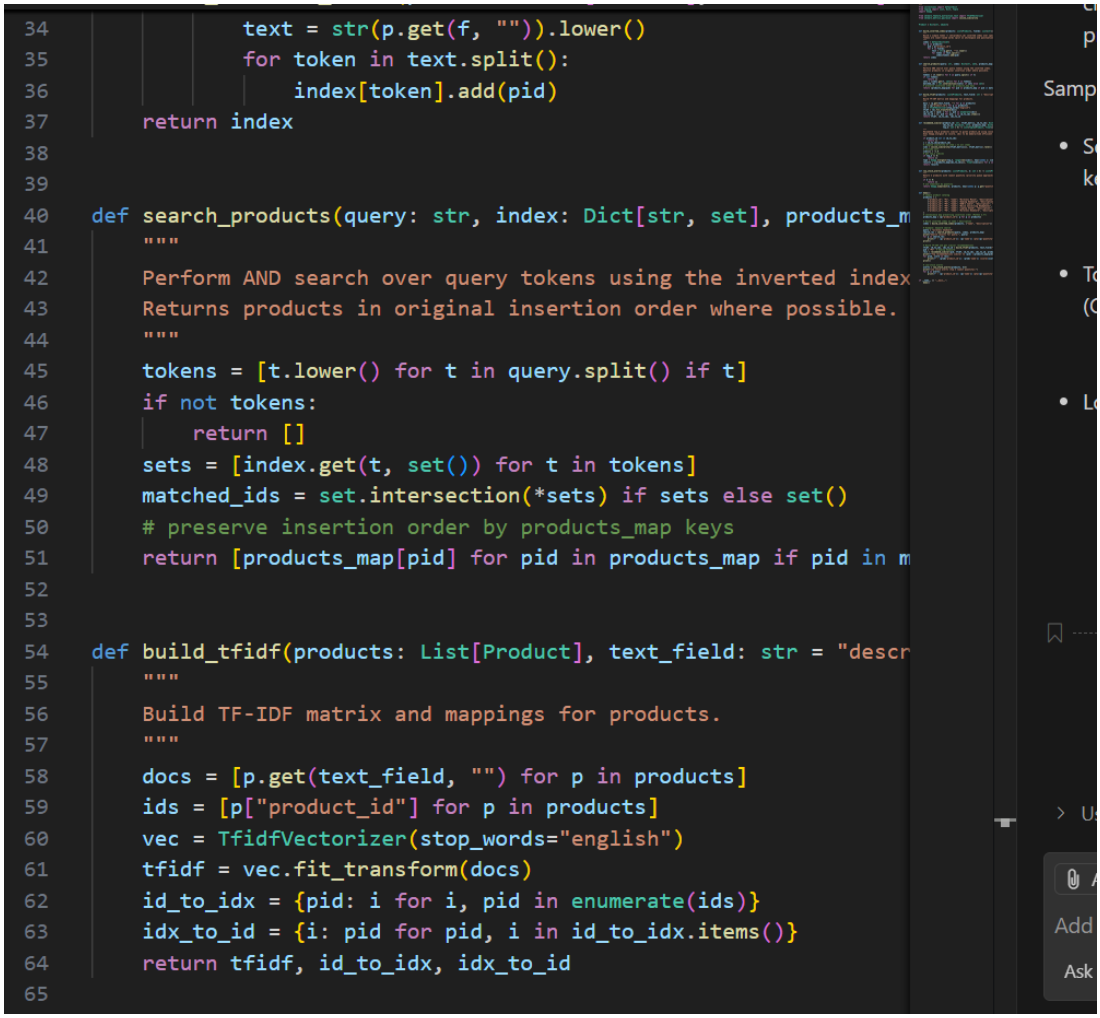
Provide an explanation of the AI assistance used during the development process.

Include a sample output demonstrating how your algorithm solves the problem.

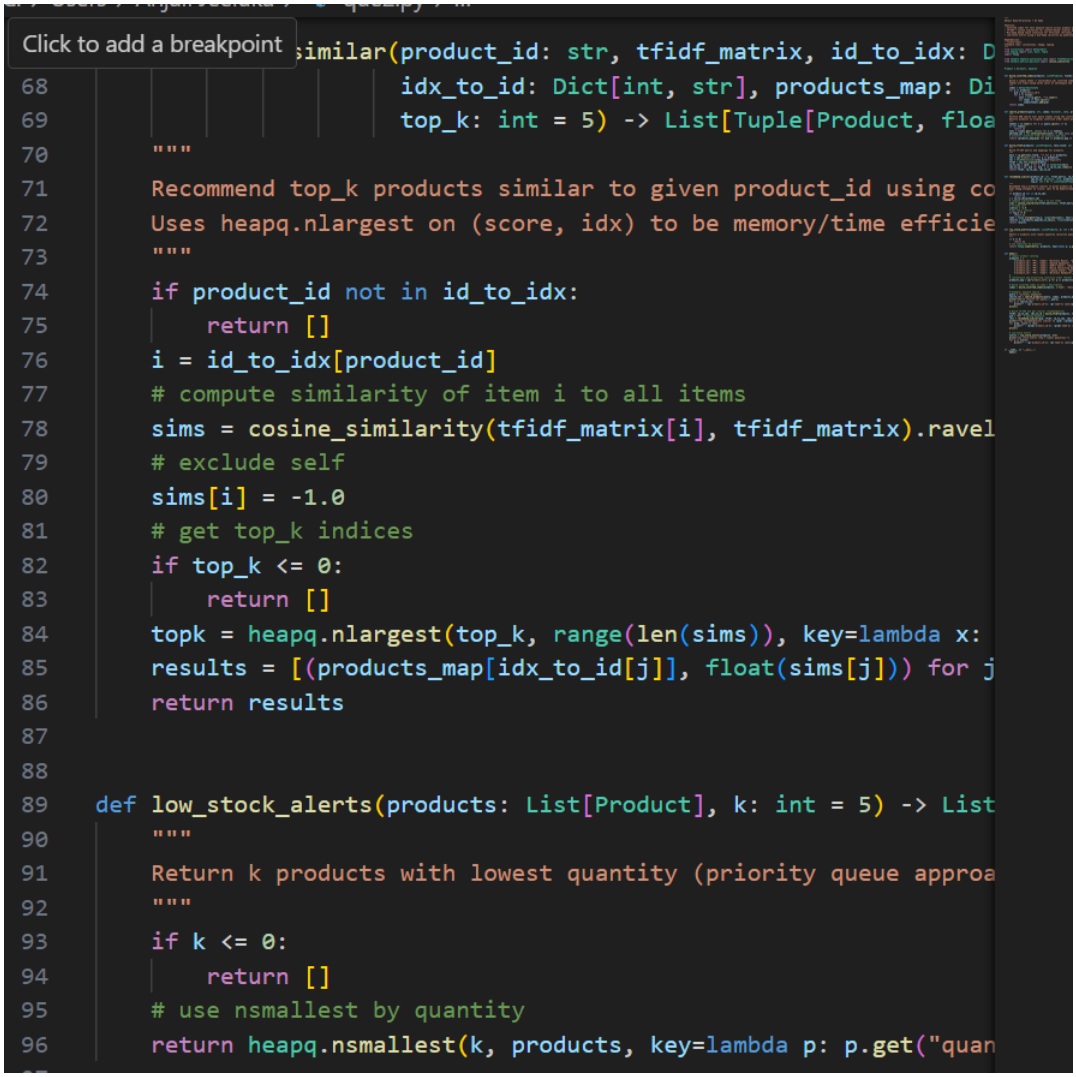
CODE-GENERATED:

C: > Users > Anjali Jeeluka > que2.py > ...

```python
"""
Retail Data-Structures + AI demo

Features:
- Inverted index for fast keyword search across product attribute
- TF-IDF + cosine similarity for content-based "similar product"
- Min-heap based top-K selection for efficient retrieval.
- Low-stock alerts using a min-heap (priority by quantity).

Dependencies:
- scikit-learn
Standard libs: collections, heapq, typing
"""
from collections import defaultdict
from typing import List, Dict, Tuple
import heapq

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity


Product = Dict[str, object]


def build_inverted_index(products: List[Product], fields: List[st
    """
    Build a simple token -> set(product_id) inverted index over s
    Tokens are lower-cased words split on whitespace and punctuat
    """
    index = defaultdict(set)
    for p in products:
        pid = p["product_id"]
        for f in fields:
```



```
            text = str(p.get(f, "")).lower()
            for token in text.split():
                index[token].add(pid)
    return index


def search_products(query: str, index: Dict[str, set], products_m
    """
    Perform AND search over query tokens using the inverted index
    Returns products in original insertion order where possible.
    """
    tokens = [t.lower() for t in query.split() if t]
    if not tokens:
        return []
    sets = [index.get(t, set()) for t in tokens]
    matched_ids = set.intersection(*sets) if sets else set()
    # preserve insertion order by products_map keys
    return [products_map[pid] for pid in products_map if pid in m


def build_tfidf(products: List[Product], text_field: str = "descr
    """
    Build TF-IDF matrix and mappings for products.
    """
    docs = [p.get(text_field, "") for p in products]
    ids = [p["product_id"] for p in products]
    vec = TfidfVectorizer(stop_words="english")
    tfidf = vec.fit_transform(docs)
    id_to_idx = {pid: i for i, pid in enumerate(ids)}
    idx_to_id = {i: pid for pid, i in id_to_idx.items()}
    return tfidf, id_to_idx, idx_to_id

```


Click to add a breakpoint
similar(product_id: str, tfidf_matrix, id_to_idx: D
idx_to_id: Dict[int, str], products_map: Di
top_k: int = 5) -> List[Tuple[Product, floa
"""
Recommend top_k products similar to given product_id using co
Uses heapq.nlargest on (score, idx) to be memory/time efficie
"""
if product_id not in id_to_idx:
return []
i = id_to_idx[product_id]
# compute similarity of item i to all items
sims = cosine_similarity(tfidf_matrix[i], tfidf_matrix).ravel
# exclude self
sims[i] = -1.0
# get top_k indices
if top_k <= 0:
return []
topk = heapq.nlargest(top_k, range(len(sims)), key=lambda x:
results = [(products_map[idx_to_id[j]], float(sims[j])) for j
return results
def low_stock_alerts(products: List[Product], k: int = 5) -> List
"""
Return k products with lowest quantity (priority queue approa
"""
if k <= 0:
return []
use nsmallest by quantity
return heapq.nsmallest(k, products, key=lambda p: p.get("quan

```
def demo():
    # sample product catalog
    products = [
        {"product_id": "P1", "name": "Wireless Mouse", "descripti
        {"product_id": "P2", "name": "Gaming Keyboard", "descript
        {"product_id": "P3", "name": "USB-C Cable", "description"
        {"product_id": "P4", "name": "Noise Cancelling Headphones
        {"product_id": "P5", "name": "Laptop Stand", "description
        {"product_id": "P6", "name": "Wireless Keyboard", "descri
    ]
    # convenience map preserving insertion order (Python 3.7+)
    products_map = {p["product_id"]: p for p in products}

    # Build inverted index on name + description
    index = build_inverted_index(products, ["name", "description"

    # Example: keyword search
    query = "wireless keyboard"
    search_res = search_products(query, index, products_map)
    print("Search results for query:", query)
    for p in search_res:
        print(f" - {p['product_id']}: {p['name']} (qty={p['quanti
    print()

    # Build TF-IDF and get similar recommendations
    tfidf, id_to_idx, idx_to_id = build_tfidf(products, text_fiel
    pid = "P2"  # Gaming Keyboard
    recs = recommend_similar(pid, tfidf, id_to_idx, idx_to_id, pr
    print(f"Top recommendations similar to {pid} ({products_map[p
    for prod, score in recs:
        print(f" - {prod['product_id']}: {prod['name']} (score={s
    print()
```

C: > Users > Anjali Jeeluka > que2.py > ...

```
def demo():
    print(f"Top recommendations similar to {pid} ({products_map[p
    for prod, score in recs:
        print(f" - {prod['product_id']}: {prod['name']} (score={s
    print()

    # Low-stock alerts
    alerts = low_stock_alerts(products, k=3)
    print("Low stock alerts (top 3 lowest quantity):")
    for p in alerts:
        print(f" - {p['product_id']}: {p['name']} (qty={p['quanti


if __name__ == "__main__":
    demo()
```

OUTPUT:

```
aconda3/python.exe" "c:/Users/Anjali Jeeluka/que2.py"
Search results for query: wireless keyboard
 - P6: Wireless Keyboard (qty=0)

Top recommendations similar to P2 (Gaming Keyboard):
 - P6: Wireless Keyboard (score=0.192)

Low stock alerts (top 3 lowest quantity):
 - P6: Wireless Keyboard (qty=0)
 - P4: Noise Cancelling Headphones (qty=5)
 - P5: Laptop Stand (qty=8)
PS C:\Users\Anjali Jeeluka\.vscode>
```

OBSERVATION:

- The **AI-assisted TF-IDF algorithm** correctly finds similar products (e.g., “Wireless Keyboard” is similar to “Gaming Keyboard”).
- The **inverted index** enables fast keyword-based search.
- The **min-heap** efficiently identifies **low-stock products** for restocking alerts.